Controllers in ASP.NET Core Web API

Asked by: Anil Kumar | Last Update: May 29, 2022

Controllers in ASP.NET Core Web API

Controllers are the core components of an ASP.NET Core application and are the entry point for handling client requests and generating responses. They act as the bridge between the client and the server, processing incoming HTTP requests, interacting with models, and returning appropriate responses to the client.

What are Controllers in ASP.NET Core Web API?

In ASP.NET Core Web API, controllers are classes that handle incoming HTTP requests sent by clients such as browsers, mobile apps, Postman, or other services and return HTTP responses. They act as the **Traffic Police** or **Gatekeeper** for your application, directing requests to the appropriate methods based on the URL and HTTP verb (GET, POST, PUT, DELETE, etc.). Controllers manage business logic, data validation, and error handling.

In MVC terms:

- Model: Data (Entities, DTOs, Domain Models)
- View: Not used in Web APIs (unlike MVC web apps)
- Controller: The control unit that receives the request, coordinates with services and models, and returns a response.

Key Roles of a Controller

Let us understand the key Roles and Responsibilities of a Controller in a Web API application.

Routing Handling

- The controller checks if a specific URL (e.g., /api/products) and verb (GET, POST, PUT, DELETE) matches a defined route.
- Routes are defined using attributes like [Route("api/[controller]")] or [HttpGet("{id}")].
- Example: GET /api/products/5 → Controller's GetProduct(int id) method.

Data Processing

- The controller receives data from the request (JSON, query parameters, or route values) and binds it to method parameters.
- Example: POST /api/products with JSON body → Controller's CreateProduct(Product product).

Calling Services

- The controller delegates business logic to services (e.g., IProductService, IOrderService) rather than handling it directly.
- This keeps the controller lightweight, maintainable, and testable.
- Example: var product = _productService.GetById(id);

Response Generation

- The controller returns HTTP responses using helper methods like Ok(), NotFound(), BadRequest(), Created(), etc.
- Example: return Ok(product); → Returns HTTP 200 OK with product data.

Error Handling

- The controller checks for errors (invalid data, missing resources, business rule violations) and returns appropriate HTTP error responses.
- Example: if (product == null) return NotFound();

Security Enforcement

- The controller checks if a user is authorized before processing the request.
- Example: [Authorize] attribute ensures only authenticated users can access the endpoint.

Example

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    [HttpGet("{id}")]
    public IActionResult GetProduct(int id)
    {
        var product = _productService.GetById(id);
        if (product == null)
            return NotFound();
        return Ok(product);
    }

    [HttpPost]
    public IActionResult CreateProduct(Product product)
    {
        if (!ModelState.IsValid)
            return BadRequest(ModelState);
        _productService.Add(product);
        return Created($"api/products/{product.Id}", product);
    }
}
```

Analogy: Think of a waiter at a restaurant. The waiter takes orders (requests) from customers (clients), delivers them to the kitchen (services), and brings back the food (responses). The waiter doesn't cook the food (business logic) or manage the inventory (data access) — that's the kitchen's job. The waiter just coordinates the process.

Key Points:

- Controllers should be lightweight and focused on routing and HTTP concerns.
- Business logic should be in services, not in controllers.
- Controllers should be testable by mocking dependent services.
- Use dependency injection to provide services to controllers.
- Follow naming conventions (e.g., ProductsController for /api/products).

Why are Controllers Important?

Controllers are important because they provide **clear separation** for request handling, business logic, and response formatting. Without controllers, your application would have no way to receive HTTP requests or return structured responses to clients.

Why are Controllers Important?

- **Separation of Concerns:** Keeps business logic out of the HTTP layer.
- **Testability:** Controllers can be unit tested by mocking services.
- **Maintainability:** Changes to routing or response formats don't affect business logic.
- **Reusability:** Multiple controllers can share the same services.
- **Clear Entry Point:** Developers know where to look for HTTP-related issues.

Creating Controllers Step by Step

Step 1: Create a Controller Class

- Inherit from ControllerBase (for API controllers) or Controller (for MVC controllers).
- Add the [ApiController] attribute to enable automatic model validation.
- Define a constructor to inject dependencies (services, logger, configuration).

Step 2: Define Routes

- Use [Route("api/[controller]")] to define the base route.
- Use [HttpGet], [HttpPost], [HttpPut], [HttpDelete] attributes to define specific endpoints.
- Example: [HttpGet("{id}")] → GET /api/products/5

Step 3: Implement Action Methods

- Create methods that handle specific HTTP requests.
- Use IActionResult or ActionResult<T> as the return type.
- Example: public IActionResult GetProduct(int id)

Step 4: Return Responses

- Use helper methods like Ok(), NotFound(), BadRequest(), Created(), etc.
- Example: return Ok(product);

Example: Complete Controller

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    private IProductService _productService;
    private ILogger<ProductsController> _logger;

    public ProductsController(IProductService productService, ILogger<ProductsController> logger)
    {
        _productService = productService;
        _logger = logger;
    }

    [HttpGet]
    public IActionResult GetAllProducts()
    {
        var products = _productService.GetAll();
        return Ok(products);
    }

    [HttpGet("{id}")]
    public IActionResult GetProduct(int id)
    {
        var product = _productService.GetById(id);
        if (product == null)
            return NotFound();
        return Ok(product);
    }

    [HttpPost]
    public IActionResult CreateProduct(Product product)
    {
        if (!ModelState.IsValid)
            return BadRequest(ModelState);
        _productService.Add(product);
        return Created($"api/products/{product.Id}", product);
    }

    [HttpPut("{id}")]
    public IActionResult UpdateProduct(int id, Product product)
    {
        if (!ModelState.IsValid)
            return BadRequest(ModelState);
        var existingProduct = _productService.GetById(id);
        if (existingProduct == null)
            return NotFound();
        _productService.Update(product);
        return NoContent();
    }

    [HttpDelete("{id}")]
    public IActionResult DeleteProduct(int id)
    {
        var product = _productService.GetById(id);
        if (product == null)
            return NotFound();
        _productService.Delete(id);
        return NoContent();
    }
}
```

Controller Lifecycle

1. Request arrives at the server.
2. Routing matches the URL to a controller and action.
3. Controller instance is created (dependency injection).
4. Action method executes.
5. Response is generated.
6. Controller instance is disposed.

Best Practices

- Keep controllers thin — business logic belongs in services.
- Use dependency injection for services and loggers.
- Return appropriate HTTP status codes (200, 201, 400, 404, 500).
- Use [ApiController] for automatic validation.
- Follow naming conventions (ProductsController for /api/products).
- Use [Authorize] for security.
- Handle errors gracefully.
- Write unit tests for controllers.

Conclusion

Controllers are the entry point for all HTTP requests in ASP.NET Core Web API. They act as the traffic police, routing requests to the appropriate methods, coordinating with services, and returning structured responses to clients. Controllers should be lightweight, focused on HTTP concerns, and testable. Proper use of controllers ensures a clean, maintainable, and scalable architecture.

Controllers in ASP.NET Core Web API

Controllers are the core components of an ASP.NET Core application and are the entry point for handling client requests and generating responses. They act as the bridge between the client and the server, processing incoming HTTP requests, interacting with models, and returning appropriate responses to the client.

What are Controllers in ASP.NET Core Web API?

In ASP.NET Core Web API, controllers are classes that handle incoming HTTP requests sent by clients such as browsers, mobile apps, Postman, or other services and return HTTP responses. They act as the **Traffic Police** or **Gatekeeper** for your application, directing requests to the appropriate methods based on the URL and HTTP verb (GET, POST, PUT, DELETE, etc.). Controllers manage business logic, data validation, and error handling.

In MVC terms:

- Model: Data (Entities, DTOs, Domain Models)
- View: Not used in Web APIs (unlike MVC web apps)
- Controller: The control unit that receives the request, coordinates with services and models, and returns a response.

Key Roles of a Controller

Let us understand the key Roles and Responsibilities of a Controller in a Web API application.

Routing Handling

- The controller checks if a specific URL (e.g., /api/products) and verb (GET, POST, PUT, DELETE) matches a defined route.
- Routes are defined using attributes like [Route("api/[controller]")] or [HttpGet("{id}")].
- Example: GET /api/products/5 → Controller's GetProduct(int id) method.

Data Processing

- The controller receives data from the request (JSON, query parameters, or route values) and binds it to method parameters.
- Example: POST /api/products with JSON body → Controller's CreateProduct(Product product).

Calling Services

- The controller delegates business logic to services (e.g., IProductService, IOrderService) rather than handling it directly.
- This keeps the controller lightweight, maintainable, and testable.
- Example: var product = _productService.GetById(id);

Response Generation

- The controller returns HTTP responses using helper methods like Ok(), NotFound(), BadRequest(), Created(), etc.
- Example: return Ok(product); → Returns HTTP 200 OK with product data.

Error Handling

- The controller checks for errors (invalid data, missing resources, business rule violations) and returns appropriate HTTP error responses.
- Example: if (product == null) return NotFound();

Security Enforcement

- The controller checks if a user is authorized before processing the request.
- Example: [Authorize] attribute ensures only authenticated users can access the endpoint.

Example

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    [HttpGet("{id}")]
    public IActionResult GetProduct(int id)
    {
        var product = _productService.GetById(id);
        if (product == null)
            return NotFound();
        return Ok(product);
    }

    [HttpPost]
    public IActionResult CreateProduct(Product product)
    {
        if (!ModelState.IsValid)
            return BadRequest(ModelState);
        _productService.Add(product);
        return Created($"api/products/{product.Id}", product);
    }
}
```

Analogy: Think of a waiter at a restaurant. The waiter takes orders (requests) from customers (clients), delivers them to the kitchen (services), and brings back the food (responses). The waiter doesn't cook the food (business logic) or manage the inventory (data access) — that's the kitchen's job. The waiter just coordinates the process.

Key Points:

- Controllers should be lightweight and focused on routing and HTTP concerns.
- Business logic should be in services, not in controllers.
- Controllers should be testable by mocking dependent services.
- Use dependency injection to provide services to controllers.
- Follow naming conventions (e.g., ProductsController for /api/products).

Why are Controllers Important?

Controllers are important because they provide **clear separation** for request handling, business logic, and response formatting. Without controllers, your application would have no way to receive HTTP requests or return structured responses to clients.

Why are Controllers Important?

- **Separation of Concerns:** Keeps business logic out of the HTTP layer.
- **Testability:** Controllers can be unit tested by mocking services.
- **Maintainability:** Changes to routing or response formats don't affect business logic.
- **Reusability:** Multiple controllers can share the same services.
- **Clear Entry Point:** Developers know where to look for HTTP-related issues.

Creating Controllers Step by Step

Step 1: Create a Controller Class

- Inherit from ControllerBase (for API controllers) or Controller (for MVC controllers).
- Add the [ApiController] attribute to enable automatic model validation.
- Define a constructor to inject dependencies (services, logger, configuration).

Step 2: Define Routes

- Use [Route("api/[controller]")] to define the base route.
- Use [HttpGet], [HttpPost], [HttpPut], [HttpDelete] attributes to define specific endpoints.
- Example: [HttpGet("{id}")] → GET /api/products/5

Step 3: Implement Action Methods

- Create methods that handle specific HTTP requests.
- Use IActionResult or ActionResult<T> as the return type.
- Example: public IActionResult GetProduct(int id)

Step 4: Return Responses

- Use helper methods like Ok(), NotFound(), BadRequest(), Created(), etc.
- Example: return Ok(product);

Example: Complete Controller

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    private IProductService _productService;
    private ILogger<ProductsController> _logger;

    public ProductsController(IProductService productService, ILogger<ProductsController> logger)
    {
        _productService = productService;
        _logger = logger;
    }

    [HttpGet]
    public IActionResult GetAllProducts()
    {
        var products = _productService.GetAll();
        return Ok(products);
    }

    [HttpGet("{id}")]
    public IActionResult GetProduct(int id)
    {
        var product = _productService.GetById(id);
        if (product == null)
            return NotFound();
        return Ok(product);
    }

    [HttpPost]
    public IActionResult CreateProduct(Product product)
    {
        if (!ModelState.IsValid)
            return BadRequest(ModelState);
        _productService.Add(product);
        return Created($"api/products/{product.Id}", product);
    }

    [HttpPut("{id}")]
    public IActionResult UpdateProduct(int id, Product product)
    {
        if (!ModelState.IsValid)
            return BadRequest(ModelState);
        var existingProduct = _productService.GetById(id);
        if (existingProduct == null)
            return NotFound();
        _productService.Update(product);
        return NoContent();
    }

    [HttpDelete("{id}")]
    public IActionResult DeleteProduct(int id)
    {
        var product = _productService.GetById(id);
        if (product == null)
            return NotFound();
        _productService.Delete(id);
        return NoContent();
    }
}
```

Controller Lifecycle

1. Request arrives at the server.
2. Routing matches the URL to a controller and action.
3. Controller instance is created (dependency injection).
4. Action method executes.
5. Response is generated.
6. Controller instance is disposed.

Best Practices

- Keep controllers thin — business logic belongs in services.
- Use dependency injection for services and loggers.
- Return appropriate HTTP status codes (200, 201, 400, 404, 500).
- Use [ApiController] for automatic validation.
- Follow naming conventions (ProductsController for /api/products).
- Use [Authorize] for security.
- Handle errors gracefully.
- Write unit tests for controllers.

Conclusion

Controllers are the entry point for all HTTP requests in ASP.NET Core Web API. They act as the traffic police, routing requests to the appropriate methods, coordinating with services, and returning structured responses to clients. Controllers should be lightweight, focused on HTTP concerns, and testable. Proper use of controllers ensures a clean, maintainable, and scalable architecture.

Previous Article: ASP.NET Core Web API Project Structure

Next Article: Models in ASP.NET Core Web API

4 thoughts on "Controllers in ASP.NET Core Web API"

Dotnet Learner
Sep 29, 2025 at 3:12 pm

Can anyone give me such course in any page of the net or if that can be sent to me as I am a student

Reply

Anonymous
Sep 29, 2025 at 3:12 pm

Thank you very much Sir

Reply

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *

Email *

Website

Post Comment

Home | Privacy Policy | Contact | ASP.NET Core | Angular Jobs | ASP.NET Core Tutorials | ASP.NET Core Interviews | ASP.NET Core Jobs | ASP.NET Core Interview Questions | ASP.NET Core Interview Questions and Answers | ASP.NET Core Interviews | ASP.NET Core Interviews | ASP.NET Core Interviews